
Final Project Proposal

Nicole Fong, Ria Kanwar, Jisoo Bu, Ezekiel Zaldivar

[team name here]

UCSD CSE25 WI26

nifong@ucsd.edu, rkanwar@ucsd.edu, j5bu@ucsd.edu, ezzaldivar@ucsd.edu

Abstract

We implemented a **Movie Recommendation System** using **collaborative filtering** on the Kaggle MovieLens 20M dataset. Our model utilizes regularized Matrix Factorization (MF) trained via **Stochastic Gradient Descent (SGD)** to learn latent representations of users and movies. To enhance results for users with limited history, we combined MF predictions with item popularity scores. Recommendations are generated by ranking predicted ratings and selecting the top 10 items. We evaluated system ranking performance with **Precision@10** as the primary ranking metric and **Root Mean Square Error (RMSE)** as a secondary metric for rating prediction accuracy. Benchmarking against random and popularity-based baselines demonstrates the model's personalization capabilities and learning efficacy.

1 Introduction

1.1 Recommender Systems

Modern online platforms such as Netflix, Amazon, and Spotify rely heavily on recommender systems to help users navigate extremely large catalogs of content. As digital libraries continue to grow, users are often faced with thousands or even millions of potential items, making it difficult to manually search for relevant options. Recommender systems address this problem by analyzing patterns in user behavior, such as ratings, purchases, or viewing history, to predict which items a user is most likely to prefer. By automatically filtering and prioritizing content based on estimated user preferences, these systems help improve user experience and increase engagement. Among the many approaches to building recommender systems, two common types are content-based methods, which rely on item attributes, and collaborative filtering methods, which learn from patterns in user behavior.

1.2 Movie Recommendation Problem

Movie recommendation involves predicting which films a user is likely to enjoy based on their past ratings. In practice, this requires estimating missing entries in the user-movie rating matrix and generating a ranked list of top recommendations for each user. However, because user-item interactions in the dataset are highly sparse, the model must learn meaningful preference patterns from only a limited amount of data. This sparsity also leads to the cold start problem, where users or movies with little or no previous interactions are difficult to model effectively.

1.3 Previous and Related Work

This project focuses on collaborative filtering methods for personalized movie recommendations using the MovieLens dataset. Unlike content-based methods that rely on explicit metadata such as genres or descriptions, collaborative filtering uses patterns in user-item interactions to identify similarities between users or items.

Collaborative filtering approaches can be broadly divided into memory-based methods such as user-user or item-item similarity, and model-based methods such as matrix factorization. Matrix factorization was popularized by the Netflix Prize competition, where it showed strong performance in large-scale rating prediction and recommendation tasks. Inspired by this approach, we implement and analyze a recommender system that uses regularized matrix factorization to learn latent features that generalize personalized top-10 recommendations.

1.4 Our Goal

To address these challenges, we implement a collaborative filtering model based on regularized matrix factorization. In this approach, users and movies are represented by latent feature vectors that capture underlying preference patterns in the data. By learning these latent representations through gradient-based optimization, the model can estimate ratings for movies that a user has not previously interacted with.

The system generates personalized recommendations by predicting ratings for unseen movies and ranking them accordingly. Model performance is evaluated using both rating prediction accuracy and ranking metrics, and compared against simple baseline methods.

2 Data Sourcing and Processing

We use the **MovieLens 20M dataset**, sourced via Kaggle from GroupLens Research. This dataset contains millions of user-provided movie ratings on a scale from 0.5 to 5.0, along with associated movie metadata such as titles and genres. It is widely used as a benchmark for evaluating recommender systems due to its scale and real-world sparsity.

However, due to computational constraints, we do not use the full dataset directly. Instead, we construct a smaller working dataset by sampling a subset of users for training while preserving their full rating histories.

2.1 Dataset Statistics & Characteristics

To understand the scope of the recommendation problem, we summarize key statistics of our sampled dataset in Table 1.

Metric	Value
Number of users	1385
Number of movies	10,463
Number of ratings	199,855
Average ratings per user	144.3
Average ratings per movie	19.1
Matrix Sparsity	98.62%

Table 1: MovieLens 20M (1% User sample) Dataset Characteristics

60 The dataset exhibits a high level of sparsity, with over 98% of the user-item matrix entries unobserved.
 61 Matrix sparsity is defined as:

$$\text{sparsity} = 1 - \frac{\text{number of observed ratings}}{\text{number of users} \times \text{number of movies}}$$

62 This means that only a very small fraction of all possible user-movie interactions are present in the
 63 data.

64 Such sparsity presents a significant challenge for recommendation systems, as the model must infer
 65 user preferences from limited data. This motivates the use of matrix factorization, which is effective
 66 at learning latent patterns in sparse datasets.

67 2.2 Data Preprocessing

68 To reduce computational costs, we sample a 1% subset of users from the MovieLens 20M dataset,
 69 retaining their full rating histories. Rather than using a uniform rating sampling, this user-based
 70 sampling preserves the architecture necessary for collaborative filtering.

71 After sampling, we split the dataset by allocating 80% of interactions to the training set and 20% for
 72 testing. This ensures that every user appears in both the training and test sets, allowing the model to
 73 learn specific preferences while evaluating on unseen data. Despite high matrix sparsity, sufficient
 74 interaction data remains for effective model training.

75 2.3 Exploratory Data Analysis

76 To better understand the structure of the dataset, we analyze the distribution of ratings, user activity,
 77 and movie popularity, as shown in Figure 1.

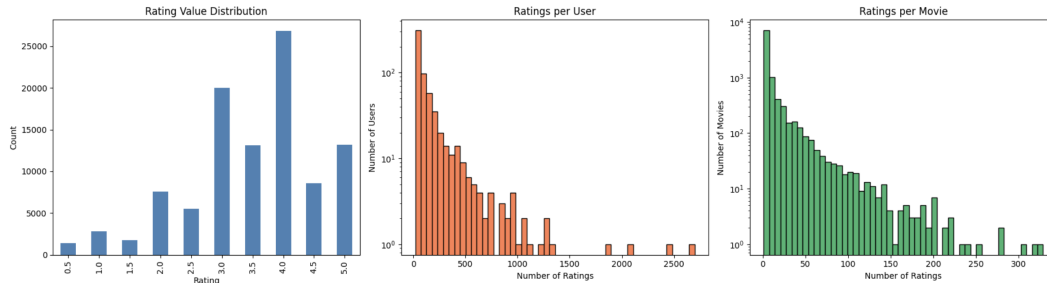


Figure 1: Distribution of rating values, ratings per user, and ratings per movie

78 As shown in Figure 1, the rating distribution is concentrated between 3.0 and 5.0, indicating a positive
 79 feedback bias. On the other hand, movie ratings exhibit a strong long-tail pattern where a few popular
 80 titles receive most interactions while the majority remain rarely rated.

81 This high sparsity and skewed distribution require the model to infer preferences from limited data
 82 without overfitting. These challenges motivate our use of Matrix Factorization, which effectively
 83 learns latent structures within such sparse interaction data.

84 To illustrate this imbalance, Table 2 lists the most frequently rated movies in the dataset:

Movie Title	No. Ratings	Avg. Rating	Genres
Forrest Gump (1994)	329	4.16	Comedy Drama Romance
Shawshank Redemption (1994)	317	4.43	Crime Drama
Pulp Fiction (1994)	307	4.20	Comedy Crime Drama Thriller
Silence of the Lambs (1991)	279	4.16	Crime Horror Thriller
Matrix, The (1999)	278	4.19	Action Sci-Fi Thriller
Star Wars: Ep. IV (1977)	251	4.23	Action Adventure Sci-Fi
Jurassic Park (1993)	238	3.75	Action Adventure Sci-Fi Thriller
Braveheart (1995)	237	4.03	Action Drama War
Terminator 2 (1991)	224	3.97	Action Sci-Fi
Schindler's List (1993)	220	4.23	Drama War

Table 2: Top 10 Most Rated Movies

3 Model Description

Our primary model uses **Matrix Factorization** to represent users (u) and movies (i) as k -dimensional latent vectors. These vectors capture underlying preferences and item characteristics. The predicted rating $\hat{r}_{ui}$ combines global bias terms with the interaction between these latent representations:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$$

where:

- $\hat{r}_{ui}$ is the predicted rating
- μ is the global mean rating
- b_u and b_i are the user and item bias terms
- p_u and q_i are the user and item latent vectors
- $p_u^\top q_i$ is the dot product between the two vectors

This formulation captures both global rating trends and personalized deviations. While bias terms account for systematic user tendencies and item popularity, latent vectors encode hidden user-item relationships. These representations allow the model to generalize beyond observed data and estimate ratings for unseen pairs.

3.1 Training Procedure

The model relies on a mix of learned and manual configurations to function. The **parameters** are the internal weights (p_u and q_i vectors) that the model discovers on its own during the training process. On the other hand, the **hyperparameters** are the settings we chose beforehand to instruct how the model learns. For our implementation, we used the following configurations based on our code:

- **Latent Factors (k): 20** is the size of the hidden feature vectors for users and movies
- **Learning Rate: 0.005** is the step size for each iteration of optimization
- **Regularization (λ): 0.05** is the penalty applied to the model, which prevents overfitting
- **Training Epochs: 20** The number of full passes over the training data.

3.2 Recommendation Strategy

The trained model is used to generate predicted ratings for user-item pairs. For users or items not present in the training set, predictions default to the global mean rating, providing a simple baseline for cold-start cases.

112 In addition to matrix factorization, we incorporate an item popularity component. Popularity is
113 computed as the normalized frequency of interactions for each movie in the training data, capturing
114 global trends in user behavior.

115 To improve recommendation quality, we use a hybrid scoring approach that combines personalized
116 predictions with item popularity:

$$\text{score} = 0.7 \cdot \hat{r}_{ui} + 0.3 \cdot \text{popularity}_i$$

117 This formulation balances individual user preferences with globally popular items, helping to mitigate
118 sparsity and improve ranking performance.

119 For each user, the system evaluates all unseen movies, ranks them according to the final score, and
120 returns the top- k highest-scoring items as recommendations.

121 4 Results and Discussion

122 We evaluate our recommender system using both rating prediction accuracy and ranking performance.
123 Specifically, we use **Root Mean Square Error (RMSE)** to measure how well the model predicts
124 user ratings, and **Precision@10** to assess the quality of the top- k recommendations.

125 To contextualize performance, we compare our model against two baselines: a random recommenda-
126 tion strategy and a popularity-based approach. The following sections present quantitative results and
127 analyze the strengths and limitations of each method.

128 4.1 Quantitative Results: Primary Performance Baseline Models

129 The primary goal of our recommender system is to provide a relevant ranked list of items. We evaluate
130 this using Precision@10, which measures the proportion of recommended items that the user actually
131 interacted with in the test set.

132 To assess the effectiveness of our approach, we compare performance across four methods:

- 133 1. **Random Baseline:** Recommends unseen movies uniformly at random, establishing a
134 "worst-case" performance if the model were to guess blindly.
- 135 2. **Popularity-Based Baseline:** Recommends the global average of the highest rated movies
136 or most frequent interactions. This represents a strong non-personalized strategy.
- 137 3. **Matrix Factorization (MF):** A personalized model that learns latent user-item relationships.
- 138 4. **MF + Popularity (Hybrid):** A combined model that integrates personalized predictions
139 with global popularity.

140 The results of this comparison are illustrated in Figure 2:

141 Random Baseline performs near zero, representing the difficulty of the recommendation task under
142 high sparsity. The Matrix Factorization model improves significantly over random recommendations,
143 demonstrating that it learns meaningful latent user-item relationships.

144 However, the Popularity Baseline achieves higher performance than the Matrix Factorization model,
145 indicating that global interaction trends provide a strong signal for recommendation in sparse datasets.
146 This indicates that, in highly sparse settings, global interaction trends can provide stronger ranking
147 signals than personalization alone.

148 The hybrid model achieves the best overall performance, reaching a Precision@10 of 0.0901. This
149 improvement demonstrates that combining personalized representations with global popularity leads
150 to more effective recommendations than either approach alone.

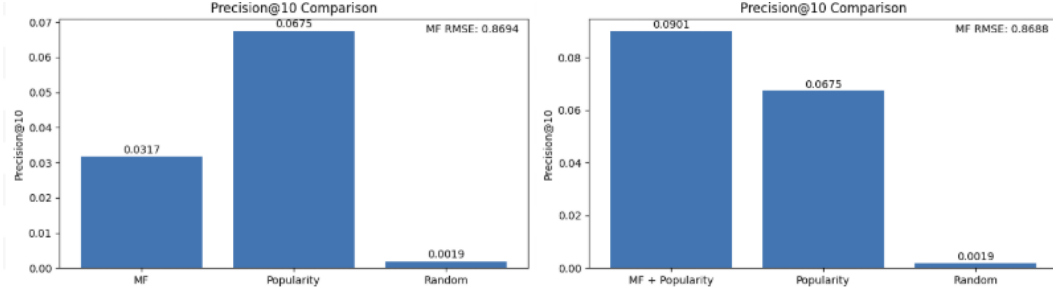


Figure 2: Comparison of Precision@10 across Random, Popularity, and Matrix Factorization, and Hybrid models.

4.2 Quantitative Results: Secondary RMSE

The Matrix Factorization model was trained for 20 epochs, reaching a final Training RMSE of approximately 0.8156 and a Test RMSE of approximately 0.8694. While the training error is lower, the gap between these metrics is expected given the 98.6% sparsity of the dataset and indicates mild overfitting rather than poor generalization.

Despite using L_2 regularization ($\lambda = 0.1$) to limit overfitting, the model naturally fits observed training interactions more closely than unseen test data. However, the test RMSE remains relatively low, suggesting that the model generalizes reasonably well despite the high sparsity of the user-item interaction matrix.

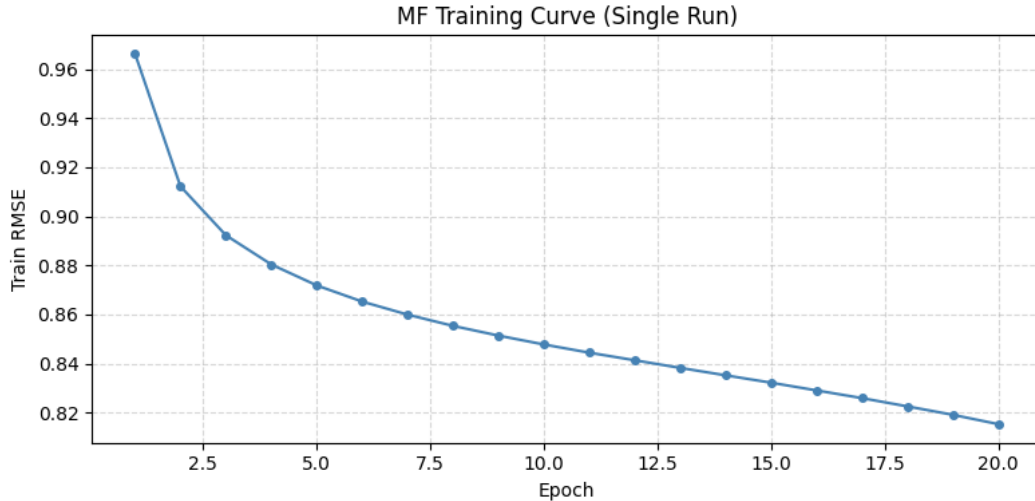


Figure 3: Training RMSE over 20 epochs for the matrix factorization model, showing steady convergence during optimization.

However, an important observation is that strong performance in RMSE does not directly translate to effective recommendation quality. For the pure matrix factorization model, although it achieves relatively low prediction error, its ranking performance (Precision@10) remains significantly lower than the popularity baseline.

This highlights a key limitation of optimizing for RMSE: the objective focuses on minimizing prediction error rather than correctly ordering items. As a result, the model may produce accurate ratings overall but fail to consistently rank the most relevant items at the top of the recommendation list.

This limitation directly motivated our transition to a hybrid approach. Since the popularity baseline already captures strong global interaction patterns, we incorporate this signal into the matrix factoriza-

tion model. The resulting MF + Popularity hybrid leverages both personalized latent representations and global trends, leading to improved ranking performance that surpasses the standalone baselines.

4.3 Qualitative Results

The Matrix Factorization model learns latent relationships by embedding users and movies into a shared space. This suggests that the model is able to group items with similar genres or tones even without explicit using genre metadata.

In practice, these learned representations allow the model to move beyond simple popularity-based recommendations and capture more nuanced user preferences. Users who consistently interact with specific genres or styles tend to receive recommendations that reflect those patterns, rather than only globally popular items. However, due to the high sparsity of the dataset, these patterns are not always strong enough to consistently dominate the ranking, which helps explain the weaker performance of pure matrix factorization in Precision@10.

These were our key results:

1. By identifying user-specific preference patterns, the model moves beyond global hits to deliver relevant and **personalized** results rather than just famous ones.
2. Titles for genres like Action and Sci-Fi titles cluster together in the latent space, suggesting that the model is able to learn preference structures despite 98.3% sparsity.
3. Despite learning meaningful structure, the model is limited by data sparsity, which restricts its ability to reliably rank less popular items. This contributes to weaker recommendation performance compared to approaches that leverage global popularity signals.

5 Conclusion

5.1 Summary

This project implemented a movie recommendation system using matrix factorization to model user-item interactions. The model successfully learned latent representations that capture underlying user preferences, achieving strong performance in terms of rating prediction accuracy (RMSE).

However, evaluation revealed that prediction accuracy alone does not guarantee effective recommendation quality. While the pure matrix factorization model achieved relatively low RMSE, it underperformed compared to a popularity-based baseline in ranking tasks (Precision@10), showing the limitations of optimizing solely for prediction error.

To address this, we introduced a hybrid approach that combines matrix factorization with item popularity. This approach leverages both personalized latent representations and global interaction patterns, resulting in improved ranking performance that surpasses both the standalone matrix factorization and baseline methods.

5.2 Limitations and Mitigations

The primary limitations of our approach were the cold-start problem and the sparsity of the dataset. Because most user-item interactions are missing, the model only has a limited amount of information to learn reliable user preferences, which weakens its ability to then rank unseen items accurately. Additionally, popular titles can skew the model, overshadowing more personal preferences.

To address these issues, we implemented two strategies:

- 209 1. **Global Mean Fallback:** When no historical data exists for a user or item, the model defaults
210 to the global average rating. This provides a reasonable baseline prediction and prevents
211 undefined behavior in cold-start scenarios.
- 212 2. **Popularity-Based Blending:** We combined Matrix Factorization predictions with a nor-
213 malized popularity signal. This stabilize recommendations in sparse data by using global
214 interaction trends while still incorporating personalized preferences.

215 6 Reflections & Contributions

216 6.1 Advice for Future Students

217 If you're interested in recommender systems, start exploring different datasets early. It's easy to just
218 pick the largest dataset you can find, but those often bring issues of high sparsity and slow training,
219 so be careful with what you're looking for. Also, make sure to set up multiple baselines because it's
220 the only way to prove your model is actually learning something useful and not just guessing.

221 AI projects like this take a ton of time, not just for the coding, but for the research needed to actually
222 explain your results. It's worth taking the time to study the math and conceptual understanding.

223 Finally, pick a topic you actually care about. These projects are a lot of work, so you might as well
224 have some fun with it!

225 6.2 Team Contributions

226 We've outlined the specific technical tasks and individual reflections for each team member:

227 Team member 1: Nicole Fong

228 Summary of tasks and contributions:

- 229 • Drafted and structured the project proposal, initial drafts, and final report using Overleaf.
- 230 • Implemented report visualizations, including tables and performance graphs.
- 231 • Analyzed quantitative and qualitative model performance as shown in the Results and
232 Discussion Section
- 233 • Refined report language for professionalism and rubric guidelines.
- 234 • Developed evaluation code, including top-k recommendations and Precision@k functions
235 for MF, Popularity, and Random baselines.
- 236 • Organized code and report for readability through structured documentation and markdown
237 cells.

238 Individual Reflection:

239 I've always been a fan of recommender systems, so getting to do this as a project really supported my
240 interests in this topic and allowed me to learn more about how user preferences really work. I learned
241 the necessity of strong evaluation to distinguish between the model learning patterns versus simply
242 remembering data.

243 There were definitely parts I missed and wish I could've expanded more on in the project report, but
244 the main reason why that was a problem was due to lack of time and team communication.

245 Team member 2: Ezekiel

246 Summary of tasks and contributions:

- 247 • Focused on testing and evaluating different model configurations, including multi-seed
248 experiments to assess performance stability.
- 249 • Contributed to analyzing results across RMSE and Precision@10, helping identify gaps
250 between prediction accuracy and ranking performance.
- 251 • Compared pure matrix factorization and hybrid approaches while deciding on model im-
252 provements.
- 253 • Debugged and validated evaluation outputs to ensure consistency across experiments.

254 Individual Reflection: This project helped me understand that strong performance on one metric
255 doesn't necessarily mean the model is actually useful. Seeing matrix factorization perform well on
256 RMSE but poorly on ranking made it clear that evaluation really depends on the task.

257 I also learned how important baselines are. The fact that a simple popularity model could outperform
258 a more complex method forced us to rethink our approach and led to a better hybrid solution.

259 Overall, I got more comfortable analyzing model behavior and thinking critically about results rather
260 than just assuming the model was working.

261 **Team member 3: Ria**

262 Summary of tasks and contributions:

- 263 • Designed and implemented the core MatrixFactorization class, including the SGD training
264 loop
- 265 • Developed visuals to communicate model performance (plot training curve and baseline
266 comparison chart)
- 267 • Contributed to analyzing results across different model implementations and baselines
- 268 • Contributed to researching and attempting different version of MF (genre-aware initialization,
269 jaccard similarity) to help choose the best approach

270 Individual Reflection:

271 I've been curious as to what factors a recommendation system uses to define each user's preferences.
272 This project helped me get an idea of how they work and how to compare them across different
273 baselines.

274 Each model can be trained with a different basis, leading to the model learning from different factors
275 and having different performance. This also leads to different levels of accuracy, helping identify
276 which factor is most important to the users of the dataset, which in this case, among rating, genres,
277 and popularity, was popularity.

278 I feel like I learned a lot of MF and recommendation systems, as well as how to analyze their results
279 and understand what factors will lead to better results.

280 **Team member 4: Jisoo**

281 Summary of tasks and contributions:

- 282 • Contributed to parts of Section 3.1 in explaining our choice for SGD and L2 regularization
 - 283 • Baseline for section 5 (conclusion and limitations)
 - 284 • Implemented code to handle cold start (Two-phase SGD training and hybrid modeling)
- 285 • Vectorized functions in `fit` and `recommend_topk_for_efficiency`

286 Individual Reflection: Overall, the project was a challenging task to complete a model and analyze
287 the factors, since I initially aimed to balance both improvement in runtime and results as well as
288 analyzing the model. Although challenging, looking at ways to mitigate cold start and struggling to
289 vectorize the functions was a good way to apply concepts. I'm sad the 7 page limit on the project
290 prevents the model from using more complex components that I have implemented initially like
291 vectorized initialization into P and Q for one-hot encoded genres, popularity biased negative sampling
292 and asymmetric regularization. Other than that, the final project was a great learning opportunity to
293 tackle real examples and the problems that come with.

294 **6.3 Generative AI**

295 While developing this project, Generative AI, specifically Gemini and Claude Code, was used for sev-
296 eral tasks. This included helping to debug our functions on the Kaggle Notebook, editing/formatting
297 sections of the final proposal, and creating visualizations (graphs, tables) to consolidate our data into
298 digestible visuals. All of our work was finalized and edited by us to ensure the project was not only
299 accurate but also followed the guidelines of the rubric and academic integrity.

300 7 References

- 301 Bennett, J., & Lanning, S. (2007). The Netflix Prize. *Proceedings of KDD Cup and Workshop 2007*, 3–
302 6. <https://www.cs.uic.edu/liub/KDD-cup-2007/proceedings/The-Netflix-Prize-Bennett.pdf>
- 303 Evidently AI. (2023, July 11). *Ranking metrics: Precision@k, Recall@k, and Mean Average*
304 *Precision (MAP) explained*. [https://www.evidentlyai.com/ranking-metrics/precision-recall-](https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k)
305 [at-k](https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k)
- 306 Ghosh, A. (2024, May 21). *Recommendation system*. LearnOpenCV.
307 <https://learnopencv.com/recommendation-system/>
- 308 Prasad, S. (2018). *MovieLens dataset - 100K ratings* [Data set]. Kaggle.
309 <https://www.kaggle.com/datasets/sriharshabspasad/movielens-dataset-100k-ratings>